

The new module syntax allows developers to include code from external JavaScript files in a manner similar to other programming languages like Ruby, Python etc.

This would allow you to write future jQuery-dependent code as:

```
import $ from "libs/jquery";
$('.element').hide();
```

If you wanted to write your own bit of JavaScript code that can be imported in other files you could do:

```
export default function (a,b) {
  // does something
}
```

You can also export named variables as `export var sheep_count = 11;`. Likewise you can import such variables `import sheep_count from "sheeplib";`.

On the server side, the Node community has already had to come up with its own system for loading external modules using the CommonJS module format, which has its own browser-equivalent: RequireJS. A number of other systems for modularising JavaScript have arisen over the past few years, but with ES6 there will finally be an official, native way built into the browser.

Classes

JavaScript uses a rather uncommon approach to Object Oriented programming, that uses prototypes, rather than classes. Even so, class-based object-oriented programming is still the most dominant paradigm, and with ES6, JavaScript is embracing that.

While ES6 classes will continue to use prototypes under the hood, developers can now use a syntax that they are more familiar with:

```
class AwesomeButton extends SimpleButton {
  constructor(width, height) {
    super(width, height);
    this.width = width;
    this.height = height;
  }
  get dimensions() {
    return {width:this.width, height:this.height};
  }
  // ...
}
```

String Interpolation and Templating

With ES6 there is a new way to declare strings that adds a number of convenient features for those dealing with long multi-line strings and those wanting to cleanly insert variable values inside strings (string interpolation).

Using backticks (``) you can easily create multi-line strings by simply typing the string across multiple lines. For example:

*pointers

>>Interesting developer news



*Future of TypeScript ECMAScript 6

>>Here's some more info on the future of ECMAScript 6 which plans to add on to JavaScript
<http://dgit.in/Nv15bt>



*Kerbal Space Program: Best mod?

>> Make your own simple programmable mod for Kerbal Space Program - kOS!
<http://dgit.in/Nv15bu>



*DIY- FrSky Programming cable

>>This complete tutorial is for making your own programming cable that can be used to program a FrSky X series receiver!
<http://dgit.in/Nv15bv>



*Best Features ECMAScript 6

>>From minor to major, this video will explain the best features of ECMAScript 6 in under three minutes
<http://dgit.in/Nv15bw>



```
let htmldoc = `<html>
<head><title>Some Title</title></
head>
<body>
</body>
</html>`;
```

The above HTML code string will be interpreted literally, i.e. line-breaks will translate to \n characters in the `htmldoc` string.

Using string interpolation we can also easily inject variables inside strings a lot more easily, for example:

```
let message = `Are you sure you want
to remove ${book} by ${author}?`;
```

Here the value of the `book` variable and the `author` variable will automatically be inserted in string at their placeholders. The bit inside the `${}` doesn't need to be a variable, it can be any JavaScript expression. For example:

```
let message = `The time is:
${formatTime(getTime().toLocaleTime('IST'))}`;
```

In the above example, the value inserted will be the result of evaluating the contents inside the `${}`.

Promises

Promises offer a much cleaner alternative to asynchronous programming than callbacks.

If you are not familiar with these terms, here is a brief. When you perform a task that will take time (for instance downloading an image), there are many ways to handle the wait. Either the entire application can halt while that task is being completed, or the application can continue responding and deal with the result of that task later.

The callback approach is that when you launch a task in the background that will take time, you also provide it with a function, called a callback function that will be sent the result of the task when it is complete. This can get convoluted quickly if you have multiple dependent tasks. For instance you want to do task1 which complete should launch task2 which should launch task3. With each callback the code becomes more confusing as you have callbacks within callbacks within callbacks. For example:

```
downloadImage("image1", function(img1) {
  downloadImage("image2", function(img2) {
    downloadImage("image3", function(img3) {
      //...
    });
  });
});
```

Promises are a different approach to the same problem. With promises the above will look like:

```
downloadImage("image1")
.then(() => downloadImage("image2"))
.then(() => downloadImage("image3")) //...
```